

Hero Hit Detection System

RFist 기반 액션 게임 피격 판정 / 히트박스 / 상태 반영 샘플 | 김철규

격투 액션 게임에서 공격 입력 이후 히트박스 활성화, 중복 피격 방지, 가드/무적/경직 판정, 데미지 결과 이벤트까지 이어지는 구조를 공개용 샘플로 정리했습니다.

HitBox`HurtBox`Guard`Stiffness`Damage Result

Attack

HitBox Open

Overlap

Resolve

Status Event

샘플 목표

- 공격 프레임 동안 히트박스 활성화
- 한 공격에서 동일 대상 중복 피격 방지
- 무적/가드/경직 상태 판정
- 데미지 결과를 UI/연출/네트워크에 이벤트로 전달

설계 기준

- HitBoxController: 공격 판정 발생
- IHurtBox: 피격 가능한 부위 계약
- HitResolver: 피격 결과 계산
- HeroStatus: HP/경직/상태 반영

히트박스과 피격 대상 계약

```
public interface IHurtBox
{
    IHitReceiver Receiver { get; }
    HurtBoxType Type { get; }
    Vector3 Center { get; }
}

public interface IHitReceiver
{
    TeamId Team { get; }
    bool IsAlive { get; }
    bool IsInvincible { get; }
    HitResult ReceiveHit(HitInfo hitInfo);
}

public enum HurtBoxType
{
    Body,
    Head,
    Guard,
    Armor
}
```

포인트

콜라이더가 어떤 캐릭터의 어떤 부위인지 직접 캐스팅하지 않고, IHurtBox를 통해 피격 가능한 계약만 사용합니다.

공격 판정 데이터

```
public readonly struct HitInfo
{
    public string AttackId { get; init; }
    public IHitReceiver Attacker { get; init; }
    public TeamId AttackerTeam { get; init; }
    public int Damage { get; init; }
    public float Stiffness { get; init; }
    public Vector3 Direction { get; init; }
    public float KnockbackPower { get; init; }
    public bool CanGuard { get; init; }
}

public readonly struct HitResult
{
    public bool Applied { get; init; }
    public bool Guarded { get; init; }
    public bool Killed { get; init; }
    public int AppliedDamage { get; init; }
    public float AppliedStiffness { get; init; }

    public static HitResult Ignored() => new() { Applied = false };
}
```

AttackId

같은 공격 프레임에서 중복 피격을 막기 위한 식별자입니다.

HitResult

피격 결과를 UI, 사운드, VFX, 네트워크 이벤트로 전달하기 위한 결과 구조체입니다.

공격 프레임 동안 히트박스 활성화

```
public sealed class HitBoxController : MonoBehaviour
{
    [SerializeField] private Collider hitCollider;
    private readonly HashSet<IHitReceiver> _hitTargets = new();
    private HitInfo _currentHitInfo;
    private bool _opened;

    public event Action<IHitReceiver, HitResult> HitApplied;

    public void Open(HitInfo hitInfo)
    {
        _currentHitInfo = hitInfo;
        _hitTargets.Clear();
        _opened = true;
        hitCollider.enabled = true;
    }

    public void Close()
    {
        _opened = false;
        hitCollider.enabled = false;
        _hitTargets.Clear();
    }

    private void OnTriggerEnter(Collider other)
    {
        if (!_opened) return;
        if (!other.TryGetComponent(out IHurtBox hurtBox)) return;
        TryApplyHit(hurtBox);
    }
}
```

실무 의도

애니메이션 이벤트나 스킬 타이밍에서 Open/Close를 호출해 공격 판정 프레임을 명확히 제어합니다.

동일 공격 중복 피격 방지

```
private void TryApplyHit(IHurtBox hurtBox)
{
    var receiver = hurtBox.Receiver;

    if (receiver == null || !receiver.IsAlive)
        return;

    if (_currentHitInfo.AttackerTeam == receiver.Team)
        return;

    if (_hitTargets.Contains(receiver))
        return;

    _hitTargets.Add(receiver);

    var result = receiver.ReceiveHit(_currentHitInfo);
    if (result.Applied)
        HitApplied?.Invoke(receiver, result);
}
```

문제

한 번의 공격에서 여러 콜라이더가 같은 캐릭터에 닿으면 데미지가 중복 적용될 수 있습니다.

해결

공격 활성화 기간 동안 피격된 `IHitReceiver`를 `HashSet`으로 보관합니다.

가드/무적/경직 판정

```
public sealed class HitResolver
{
    public HitResult Resolve(HitInfo hit, HeroStatus status)
    {
        if (status.IsDead || status.IsInvincible)
            return HitResult.Ignored();

        if (status.IsGuarding && hit.CanGuard)
        {
            status.ApplyGuardStiffness(hit.Stiffness * 0.35f);
            return new HitResult
            {
                Applied = true,
                Guarded = true,
                AppliedDamage = 0,
                AppliedStiffness = hit.Stiffness * 0.35f
            };
        }

        status.ApplyDamage(hit.Damage);
        status.ApplyStiffness(hit.Stiffness);
        status.ApplyKnockback(hit.Direction, hit.KnockbackPower);

        return new HitResult
        {
            Applied = true,
            Guarded = false,
            Killed = status.IsDead,
            AppliedDamage = hit.Damage,
            AppliedStiffness = hit.Stiffness
        };
    }
}
```

피격 결과를 캐릭터 상태에 반영

```
public sealed class HeroStatus : MonoBehaviour, IHitReceiver
{
    [SerializeField] private int maxHp = 100;
    private int _hp;
    private float _stiffness;

    public event Action<int> HpChanged;
    public event Action<HitResult> Damaged;
    public event Action Died;

    public bool IsAlive => _hp > 0;
    public bool IsDead => _hp <= 0;
    public bool IsInvincible { get; private set; }
    public bool IsGuarding { get; private set; }

    public HitResult ReceiveHit(HitInfo hitInfo)
    {
        var result = _resolver.Resolve(hitInfo, this);
        if (!result.Applied) return result;

        Damaged?.Invoke(result);
        if (result.Killed) Died?.Invoke();
        return result;
    }

    public void ApplyDamage(int damage)
    {
        _hp = Mathf.Max(0, _hp - damage);
        HpChanged?.Invoke(_hp);
    }
}
```

리뷰 포인트

피격 판정은 HitBox에서 시작되지만, 최종 상태 변화는 캐릭터 상태 객체가 책임집니다.

애니메이션 이벤트와 히트박스 타이밍

```
public sealed class AttackAnimationEventBridge : MonoBehaviour
{
    [SerializeField] private HitBoxController hitBox;
    private HitInfo _reservedHitInfo;

    public void Reserve(HitInfo hitInfo)
    {
        _reservedHitInfo = hitInfo;
    }

    // Animation Event
    public void AnimEvent_OpenHitBox()
    {
        hitBox.Open(_reservedHitInfo);
    }

    // Animation Event
    public void AnimEvent_CloseHitBox()
    {
        hitBox.Close();
    }
}
```

장점

공격 모션의 실제 타격 프레임과 판정 활성화를 맞출 수 있습니다.

주의점

네트워크 게임에서는 애니메이션 이벤트와 시뮬레이션 tick의 기준 차이를 관리해야 합니다.

피격 결과 이벤트 전파

```
public sealed class HitEventBroadcaster : NetworkBehaviour
{
    [SerializeField] private HitBoxController hitBox;

    private void OnEnable()
    {
        hitBox.HitApplied += OnHitApplied;
    }

    private void OnDisable()
    {
        hitBox.HitApplied -= OnHitApplied;
    }

    private void OnHitApplied(IHitReceiver receiver, HitResult result)
    {
        RPC_PlayHitFeedback(receiver.NetworkId, result.Guarded, result.Killed);
    }

    [Rpc(RpcSources.StateAuthority, RpcTargets.All)]
    private void RPC_PlayHitFeedback(NetworkId receiverId, bool guarded, bool killed)
    {
        // hit stop, VFX, SFX, UI feedback
    }
}
```

면접 포인트

데미지 판정과 피드백 재생을 분리합니다. 판정은 권한 있는 쪽에서 처리하고, 연출은 RPC/이벤트로 전달합니다.

테스트 가능성과 면접 어필 포인트

테스트 항목

- 동일 공격 중복 피격 방지
- 아군 피격 제외
- 무적 상태 무시
- 가드 시 데미지 0 처리
- 사망 이벤트 1회 발생
- 히트박스 Open/Close 타이밍

어필 포인트

- 액션 게임 판정 구조 이해
- 히트박스/히트박스 분리
- 상태 반영과 연출 이벤트 분리
- 네트워크 피드백 전파 고려
- 격투/액션 게임 시스템 확장 가능

연락처

Portfolio: <https://cosmicgiantkoala.github.io> Email: cosmicgiantkoala@gmail.com